

SVEUČILIŠTE U SPLITU
SVEUČILIŠNI ODJEL ZA STRUČNE STUDIJE
Stručni diplomski studij Primijenjeno računarstvo

BitWatch

Kriptovalute

Student: Ante Aljinović

Datum: 15.01.2026.

Sadržaj

1.	Opis projekta i korištene tehnologije	2
2.	Sučelje	4
3.	Popis korištenih API poziva.....	5
4.	Izvadci iz programskog koda.....	11

Opis projekta i korištene tehnologije

Projekt predstavlja web aplikaciju namijenjenu pregledavanju i analizi podataka **Bitcoin lanca blokova** (engl. *blockchain*) u stvarnom vremenu. Aplikacija korisnicima omogućuje pretraživanje i pregled osnovnih blockchain entiteta, uključujući blokove, transakcije i Bitcoin adrese, kroz jednostavno i intuitivno web sučelje.

Korisnik može pretraživati blockchain unosom broja bloka, hasha bloka, identifikatora transakcije ili Bitcoin adrese. Ovisno o vrsti upita, aplikacija dohvaća i prikazuje relevantne podatke, među ostalim veličinu i status bloka, detalje transakcije (ulaze, izlaze, naknade i pripadni blok) te informacije o adresi, kao što su saldo, ukupno primljena i poslana sredstva, zapisi nepotrošenih izlaza transakcija i nedavne transakcije.

Aplikacija također sadrži “mempool” sekciju, u kojoj je vizualizirano trenutno stanje memorijskog bazena transakcija (engl. *Mempool*) (prikaz ograničen do 100 transakcija radi performansi aplikacije) pomoću grafičkih prikaza te dodatnih detalja o transakcijama koje još nisu potvrđene u bloku. Uz to, korisnik može provjeriti trenutnu vrijednost jedne od deset odabranih kriptovaluta u njihovim novčanim protuvrijednostima, kao i pregled kretanja cijena u posljednjih 30 dana.

Cilj projekta je demonstrirati razumijevanje rada Bitcoin blockchaina te obradu stvarnih blockchain podataka.

Na serverskoj strani korišten je **Node.js** kao izvršno okruženje, uz **Express.js** radni okvir (engl. *Framework*) za izradu REST API-ja i obradu HTTP zahtjeva. Poslužiteljski dio aplikacije izravno komunicira s **Bitcoin Core** čvorom putem RPC sučelja, koristeći standardne Bitcoin RPC metode za dohvat podataka o blokovima, transakcijama, adresama i mempoolu. Implementirana je dodatna logika za izračun vrijednosti ulaza i izlaza transakcija, naknada, vremena i visine bloka te obradu coinbase transakcija.

Na klijentskoj strani korištena je **React** biblioteka za izgradnju korisničkog sučelja temeljenog na komponentama. Navigacija unutar aplikacije ostvarena je pomoću **React Router**a, dok je izgled i responzivnost sučelja implementirana korištenjem **Tailwind CSS**

frameworka. Podaci se dohvaćaju asinkrono putem HTTP zahtjeva prema backend API-ju te se dinamički prikazuju korisniku.

Za dohvat tržišnih podataka o kriptovalutama korišten je javno dostupni **CoinGecko** API, koji omogućuje prikaz trenutnih cijena i povijesnih podataka bez potrebe za autentifikacijom.

Arhitektura aplikacije temelji se na client–server modelu, gdje serverski i klijentski slojevi imaju jasno odvojene odgovornosti, dok Bitcoin Core čvor služi kao izvor stvarnih blockchain podataka.

JavaScript je programski jezik korišten za razvoj serverskog i klijentskog dijela aplikacije, omogućuje asinkronu obradu podataka i rad u stvarnom vremenu.

Node.js je izvršno okruženje koje omogućuje pokretanje JavaScript koda na serverskoj strani.

Express.js je minimalistički web framework za izradu REST API-ja i obradu HTTP zahtjeva.

Bitcoin Core je implementacija Bitcoin protokola koja omogućuje izravan pristup blockchain podacima putem RPC sučelja.

React je JavaScript biblioteka za izgradnju dinamičnih korisničkih sučelja temeljenih na komponentama.

React Router je biblioteka za upravljanje klijentskom navigacijom unutar React aplikacije.

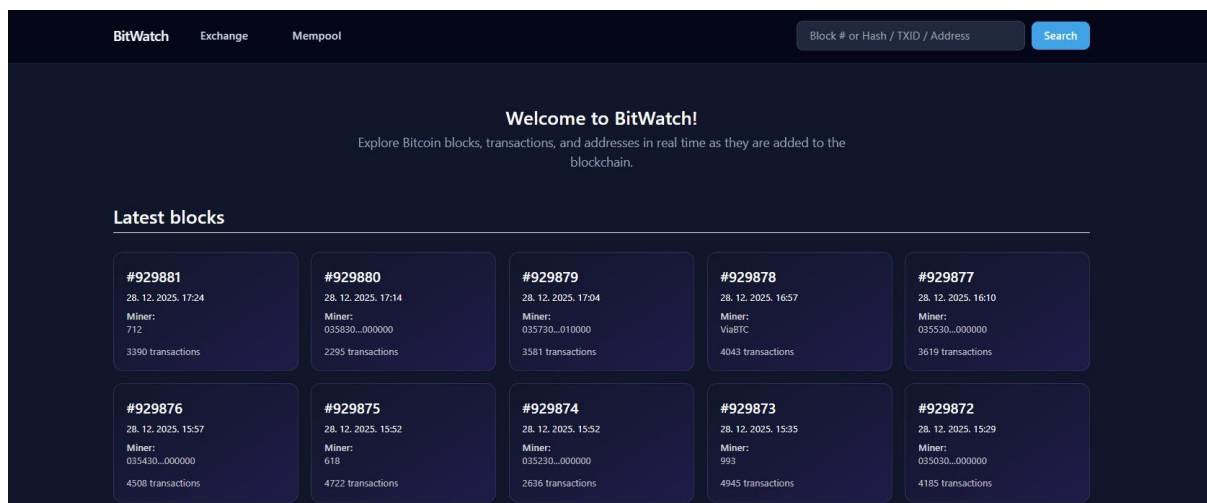
Tailwind CSS je CSS (Cascading Style Sheets) radni okvir koji omogućuje brzo i responzivno oblikovanje korisničkog sučelja.

CoinGecko API je vanjski servis za dohvat tržišnih podataka o kriptovalutama, uključujući cijene i povijesne trendove.

Sučelje

Početni zaslon aplikacije, prikazan na **slici 1**, prikazuje osnovno korisničko sučelje za pregled Bitcoin blockchaina. U gornjem dijelu nalazi se navigacijska traka s nazivom aplikacije, poveznicama prema sekcijama „Exchange“ i „Mempool“ te poljem za pretraživanje koje omogućuje unos broja bloka, hasha bloka, identifikatora transakcije ili Bitcoin adrese.

Središnji dio zaslona sadrži „Latest blocks“ sekciju s prikazom najnovije izrudarenih Bitcoin blokova. Svaki blok prikazan je u obliku kartice s osnovnim informacijama, uključujući visinu bloka, vrijeme rudarenja, identifikaciju rudara i broj transakcija u bloku.



Slika 1: Početni zaslon

Dohvat najnovijih blokova implementiran je korištenjem **Server-Sent Events (SSE)** tehnologije, koja omogućuje kontinuirani prijenos podataka sa servera prema klijentu. Umjesto dohvaćanja svih blokova odjednom, poslužiteljski dio aplikacije emitira informacije o novim blokovima čim postanu dostupni.

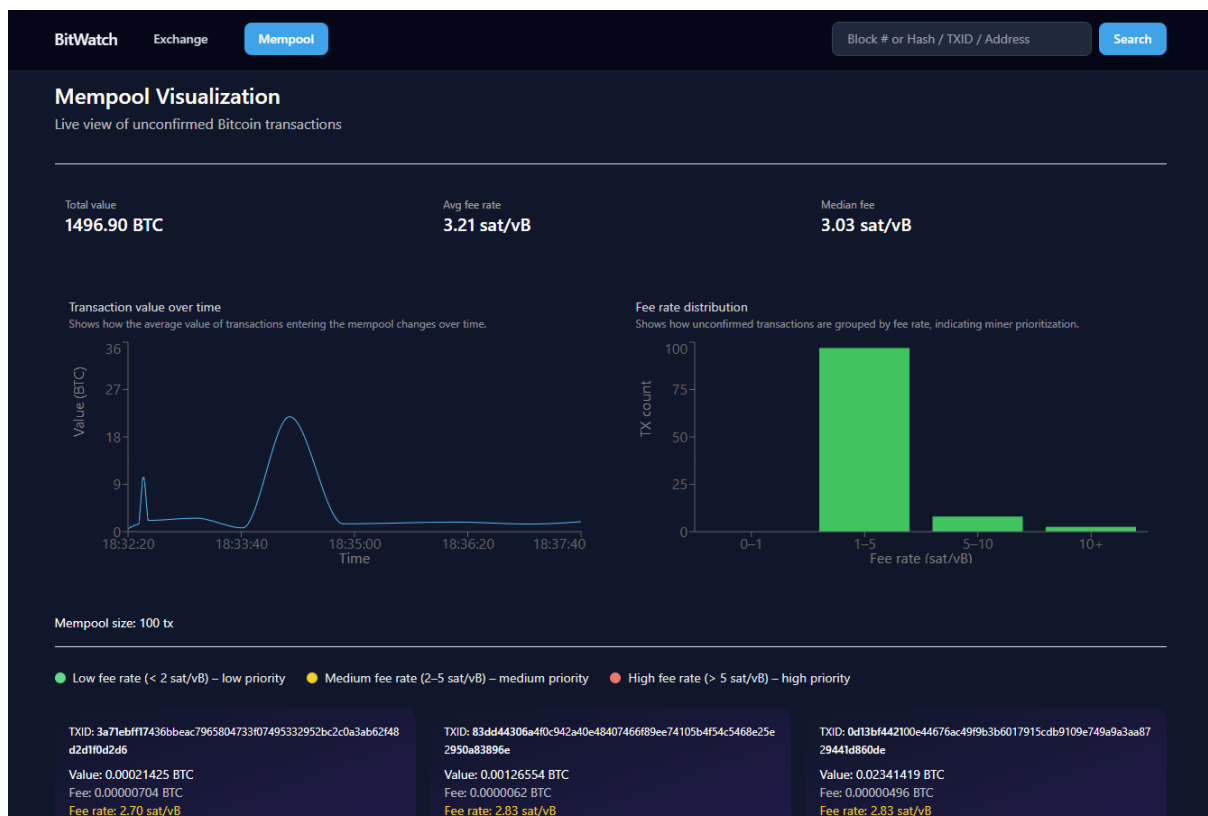
Ovakav pristup smanjuje vrijeme čekanja, poboljšava korisničko iskustvo i omogućuje ažuriranje prikaza bez ponovnog učitavanja stranice, što je posebno važno za praćenje dinamičkih podataka poput blockchaina.



Slika 2: „Exchange“ stranica

Na **slici 2** je prikazana „Exchange“ sekcija koja omogućuje korisniku pregled trenutne vrijednosti odabrane kriptovalute u odnosu na fiat valutu. Korisnik može odabrati željene valute, nakon čega se prikazuje trenutna cijena te linijski graf kretanja cijene u posljednjih 30 dana.

Podaci o cijenama dohvaćaju se putem CoinGecko API-ja i služe kao informativni dodatak osnovnim blockchain funkcionalnostima aplikacije.



Slika 3: Mempool sekcija

„Mempool“ sekcija, prikazana na **slici 3**, prikazuje trenutno stanje Bitcoin mempoola, odnosno skup nepotvrđenih transakcija koje još nisu uključene u blok. U gornjem dijelu prikazani su sažeti statistički podaci, uključujući ukupnu vrijednost transakcija u mempoolu te prosječnu i medijalnu naknadu izraženu u **sat/vB** (satoshi po virtualnom bajtu).

Središnji dio stranice sadrži grafičke prikaze koji vizualiziraju promjene vrijednosti transakcija kroz vrijeme te raspodjelu transakcija prema visini naknade, čime se prikazuje prioritet obrade od strane rudara. U donjem dijelu prikazan je popis pojedinačnih transakcija iz mempoola s osnovnim podacima, kao što su identifikator transakcije, vrijednost i naknada.

Ova stranica omogućuje korisniku vizualizaciju opterećenja mreže i trenutnih uvjeta za potvrdu transakcija u stvarnom vremenu. Zbog performansi, broj prikazanih transakcija je ograničen na 100, iako se transakcije kontinuirano mijenjaju kako ulaze i izlaze iz mempoola.

BitWatch

Exchange

Mempool

Block # or Hash / TXID / Address

Search

Transaction Details

Transaction

TXID	db767a33b1c06665a738f25247898606fc77af469b4748c7a95706601e49609a
Block	#929883
Date	28. 12. 2025. 17:29:18
Status	Confirmed (1)
Size	580 bytes
Inputs	3
Outputs	4
Input value	203.052 sat · 0.00203052 BTC · €121,83
Output value	202.552 sat · 0.00202552 BTC · €121,53
Transaction fee	500 sat · 0.00000500 BTC · €0,30
Fee rate	1.5 sat/vB
Version	2
SegWit	Yes

Inputs: 3

Input #0

0.00095336 BTC

View address

Source transaction (TXID):

0ace98eb574aeb26c5b9deed64708eb123b55fa47bdec9ec9e615686055f5b0

Address:

bc1q9xq7rs278tcnpk2hm6jzr4twl82kmclwww60t

Input #1

0.00005209 BTC

View address

Source transaction (TXID):

b1a14883dac798fb06bd005dc442b49fe867ae8c5ce15c7a591c60bbf517e9de

Address:

bc1q4775smumunq2m0ql3hmanhz6f8kj66hhkag

Outputs: 4

Output #0

0.00074803 BTC

View address

Address:

bc1qzaej5ey0qtmkk7mvpuz2c70y5y2y2y7cp70i3

Output #1

0.00020283 BTC

View address

Address:

bc1qnljism8ssmyeqft2a4dpl3dru2pjw2gkt59qu

Output #2

0.00020283 BTC

View address

Address:

bc1qeqlozcdg4z0lhp45r552yp0xqf2f764nc5nqxr

Slika 5: Detalji o transakciji

Stranica „Transaction Details“, prikazana na **slici 5**, donosi prikaz detaljnih informacija o odabranoj Bitcoin transakciji. U lijevom dijelu zaslona nalaze se osnovni podaci o transakciji, uključujući TXID, pripadni blok, vrijeme potvrde, status, veličinu, broj ulaza i izlaza, ukupnu ulaznu i izlaznu vrijednost, iznos naknade itd.

Desni dio zaslona podijeljen je na prikaz ulaza (engl. *Inputs*) i izlaza (engl. *Outputs*) transakcije. Za svaki ulaz i izlaz prikazana je pripadajuća vrijednost te Bitcoin adresa, uz mogućnost navigacije na detalje pojedine adrese. Ovakav prikaz omogućuje jasnu analizu toka sredstava unutar transakcije i njezine povezanosti s drugim blockchain entitetima.

BitWatch

Exchange

Mempool

Block # or Hash / TXID / Address

Search

Address details

bc1pnaaa0kr5usw7y5fq19stw4ecxam2ysm3k0prrr5arvs4ttkmuvqz5zryk

Balance

0,00001092 BTC

Total received

0,00231967 BTC

Total sent

0,00230875 BTC

Transactions

6

Unspent outputs (UTXOs)

938e680494c9fb6aaba8c0c892a97a442be4c44803f067bd3a313208ab5001a9:0	0,00000546 BTC
75dc923b8d473ba2e495ed4818693934af642bad46e44f8dc9e069a39eb8d85d:0	0,00000546 BTC

Recent transactions

75dc923b8d473ba2e495ed4818693934af642bad46e44f8dc9e069a39eb8d85d	Confirmed
1f73c745924543c743d756f097484258a575af2f5c5faf04e4fb8b14648b0480	Confirmed
accb29a31c2acb03b37ea12d65df8d4149bd90bf113fe10f83470d8e83518a88	Confirmed
938e680494c9fb6aaba8c0c892a97a442be4c44803f067bd3a313208ab5001a9	Confirmed

Slika 6: Detalji o adresi

Stranica „Address details“, prikazana na **slici 6**, omogućuje pregled podataka vezanih uz pojedinu Bitcoin adresu. U gornjem dijelu prikazana je sama adresa, dok su ispod nje sažeto prikazane ključne informacije, uključujući trenutni saldo, ukupno primljenu i poslanu količinu Bitcoina te ukupan broj transakcija u kojima je adresa sudjelovala.

U središnjem dijelu prikazan je popis nepotrošenih izlaza (engl. *Unspent transaction outputs (UTXO)*) povezanih s adresom, koji predstavljaju raspoloživa sredstva koja još nisu iskorištena u nekoj transakciji. Svaki *UTXO* prikazan je zajedno s pripadajućom vrijednošću.

U donjem dijelu zaslona nalazi se popis nedavnih transakcija povezanih s adresom, uz prikaz statusa svake transakcije. Ovakav prikaz omogućuje korisniku praćenje povijesti aktivnosti adrese te analizu raspoloživih i potrošenih sredstava.

Popis korištenih API poziva

Tablica 1: Popis korištenih poziva prema sučelju Bitcoin Core

Poziv	Funkcija
getBlock	Korišten sa zastavicom verbosity=2, vraća objekt s informacijama o određenom bloku te puni objekti transakcija
getRawTransaction	Vraća sirove podatke transakcije. Korišten sa zastavicom verbose=1.
getBlockStats	Vraća statistiku za blok.
getBlockHash	Vraća hash bloka u najboljem blockchainu na pruženoj visini.
getBlockCount	Vraća visinu najopterećenijeg potpuno validiranog lanca tj. najrecentniji blok.
getRawMempool	Vraća popis svih nepotvrđenih transakcija koje se trenutno nalaze u mempoolu Bitcoin čvora.
getMempoolEntry	Vraća detaljne informacije o pojedinoj transakciji u mempoolu, uključujući naknadu, veličinu i podatke potrebne za izračun fee rate-a.

Izvadci iz programskog koda

U ovom poglavlju će biti izdvojeni **najbitniji** dijelovi kôda aplikacije. U **ispisu 1** prikazana je **inicijalizacija** Bitcoin RPC API klijenta. Varijable okruženja iščitavaju se iz **.env datoteke**.

```
require("dotenv").config();
const Client = require("bitcoin-core");

const client = new Client({
  host: `http://${process.env.RPC_HOST}:${process.env.RPC_PORT}`,
  username: process.env.RPC_USER,
  password: process.env.RPC_PASS,
});

module.exports = client;
```

Ispis 1: Inicijalizacija klijenta za pristup Bitcoin RPC API-ju

U **ispisu 2** prikazana je funkcija za dohvat i slanje podataka o bloku u stvarnom vremenu. Funkcija dohvaća hash bloka na temelju visine bloka, zatim preuzima detaljne informacije o bloku te iz coinbase transakcije izdvaja podatke o rudaru. Pripremljeni podaci šalju se klijentu korištenjem Server-Sent Events mehanizma.

```
const sendBlock = async (height) => {
  const hash = await client.getBlockHash(height);
  const block = await client.getBlock(hash, 2);

  const coinbaseHex = block.tx[0]?.vin?.[0]?.coinbase ?? null;
  const minerTag = parseCoinbaseTag(coinbaseHex);

  const payload = {
    height,
    hash,
    time: block.time,
    txCount: block.tx.length,
    minerTag,
    minerCoinbase: coinbaseHex
  };
};
```

```
res.write(`data: ${JSON.stringify(payload)}\n\n`);  
};
```

Ispis 2: Dio funkcije `sendBlock()`

U **ispisu 3** prikazan je dio funkcije koji dohvaća trenutnu cijenu kriptovalute putem CoinGecko API-ja. Podaci se dohvaćaju HTTP GET zahtjevom, obrađuju u JSON formatu te se u slučaju uspjeha vraćaju klijentu, dok se u slučaju pogreške vraća odgovarajući statusni kod.

```
try {  
  const r = await fetch(  
    `https://api.coingecko.com/api/v3/simple/price?ids=${fromId}&vs_curre  
ncies=${to}`  
  );  
  const data = await r.json();  
  res.json(data);  
} catch {  
  res.status(500).json({ error: "price fetch failed" });  
}
```

Ispis 3: Dio funkcije dohvaćanja cijene kriptovalute

Ispis 4 prikazuje logiku za pretragu bloka na temelju njegove visine. Nakon provjere valjanosti unosa i trenutne visine blockchaina, dohvaća se hash bloka te detaljni podaci o bloku i pripadajuća statistika, koji se zatim šalju klijentu u strukturiranom obliku.

```
if (/^\d+$/.test(q)) {  
  const height = Number(q);  
  const tip = await client.getBlockCount();  
  
  if (height > tip) {  
    return res.status(404).json({ error: "Block not found" });  
  }  
  
  const hash = await client.getBlockHash(height);  
  const block = await client.getBlock(hash, 2);  
  const stats = await client.getBlockStats(height);  
  
  return res.json(buildBlockResponse(block, stats));  
}
```

```
}
```

Ispis 4: Pretraga bloka po visini bloka

U **ispisu 5** prikazana je pretraga bloka pomoću hasha bloka. Funkcija pokušava dohvatiti blok koristeći zadani hash te, ako uspije, dodatno dohvaća statističke podatke o bloku prije slanja odgovora klijentu.

```
if (/^[a-fA-F0-9]{64}$/.test(q)) {
  try {
    const block = await client.getBlock(q, 2);

    if (block?.hash === q) {
      const stats = await client.getBlockStats(block.height);
      return res.json(buildBlockResponse(block, stats));
    }
  } catch {}
}
```

Ispis 5: Pretraga bloka po hashu bloka

U **ispisu 6** prikazana je logika za dohvat detalja transakcije na temelju njezina identifikatora (TXID). Funkcija dohvaća sirove podatke transakcije, uz mogućnost korištenja pripadnog bloka, te u slučaju neuspjeha vraća poruku o nepostojecij transakciji.

```
if (/^[a-fA-F0-9]{64}$/.test(q)) {
  let fullTx;

  try {
    fullTx = blockFromQuery
      ? await client.getRawTransaction(q, true, blockFromQuery)
      : await client.getRawTransaction(q, true);
  } catch {
    return res.status(404).json({ error: "Not found" });
  }
  ...
}
```

Ispis 6: Pretraga transakcije po identifikatoru transakcije

U **ispisu 7** prikazana je funkcija za dohvat osnovnih informacija o Bitcoin adresi. Nakon provjere formata adrese, koristi se RPC metoda „`getAddressInfo`“ za dohvat podataka o adresi adrese, a dobiveni podaci vraćaju se klijentu u JSON formatu.

```
if (
  /^(bc1|[13])[a-zA-HJ-NP-Z0-9]{25,62}$/.test(q)
) {
  try {
    const info = await client.command("getaddressinfo", q);

    return res.json({
      type: "address",
      address: q,
      isValid: info.isValid,
      isScript: info.isScript,
      isWitness: info.isWitness,
      witnessVersion: info.witness_version ?? null,
      witnessProgram: info.witness_program ?? null
    });
  } catch {
    return res.status(404).json({ error: "Address not found" });
  }
}
```

Ispis 7: Pretraga informacija o adresi

Ispis 8 prikazuje popis svih nepotvrđenih transakcija iz Bitcoin mempoola pomoću RPC metode `getRawMempool()`. Za svaku transakciju provjerava se je li već obrađena, nakon čega se dohvaćaju detalji transakcije te dodatni podaci o naknadi i veličini iz mempoola. Ovi podaci koriste se za daljnju analizu i prijenos informacija klijentskoj strani u stvarnom vremenu.

```
try {
  mempool = await client.getRawMempool();
} catch {
  await new Promise(r => setTimeout(r, 1000));
  continue;
}
```

```
for (const txid of mempool) {  
  if (!alive) break;  
  if (seen.has(txid)) continue;  
  
  seen.add(txid);  
  
  try {  
    const tx = await client.getRawTransaction(txid, true);  
    const entry = await client.getMempoolEntry(txid);  
  
    ...  
  }  
}
```

Ispis 8: Mempool pretraga